

menler

Your turning point in the AI era.

AI APTITUDE QUESTION BANK

RAG and Knowledge Systems

About This Question Bank

These questions cover retrieval-augmented generation and enterprise knowledge infrastructure: vector databases, chunking strategies, retrieval mechanics, hallucination prevention, and production knowledge system design.

5

Sets

15

Questions per Set

75

Total Questions

SET GUIDE

Set 1 · Vector Databases and Embeddings

Vector databases and embeddings — HNSW, cosine similarity, ANN search

Set 2 · Chunking and Indexing

Chunking and indexing — strategies, overlap, hierarchical, content-aware

Set 3 · Retrieval

Retrieval — top-k, hybrid, HyDE, multi-query, re-ranking

Set 4 · Grounding and Hallucination Prevention

Grounding and hallucination prevention — faithfulness, citation, abstention

Set 5 · Enterprise Knowledge Systems

Enterprise knowledge systems — access control, freshness, observability

Each question has one correct answer. Select the best answer from the options provided.

Correct answers and explanations follow each question.

SET

1

Vector Databases and Embeddings

Set 1 of 5 · 15 Questions · RAG and Knowledge Systems

Q1. What is a vector database and why is it used in AI applications?

- A. A database that stores model weights and configuration files for fast model loading.
- B. A database optimised for storing and searching high-dimensional embedding vectors using approximate nearest-neighbour algorithms.**
- C. A relational database with a vector data type added for storing numerical arrays.
- D. A key-value store where AI-generated responses are cached for fast retrieval.

Correct Answer: B

Explanation

Vector databases (Pinecone, Weaviate, Chroma, pgvector) are built for one thing classical databases do poorly: finding the most semantically similar items in a high-dimensional space. They power semantic search and RAG by quickly finding the embedding vectors closest to a query vector — an operation called approximate nearest-neighbour (ANN) search.

Q2. What is the difference between storing embeddings in a vector database versus a traditional relational database?

- A. Vector databases store text; relational databases store numbers — embeddings are numbers so they belong in relational databases.
- B. Vector databases are schema-less; relational databases require a fixed schema that embeddings do not fit.
- C. Vector databases are only suitable for development; production systems must use relational databases for reliability.
- D. Vector databases are optimised for ANN search with specialised indexing (HNSW, IVF); relational databases support exact match and range queries but cannot efficiently search high-dimensional vectors.**

Correct Answer: D

Explanation

Relational databases can store embedding arrays but lack the specialised indexes (HNSW, IVF-flat, ScaNN) needed to search millions of vectors in milliseconds. A PostgreSQL table with a vector column (pgvector) is fine for small datasets; dedicated vector databases provide the indexing, filtering, and performance needed at scale.

Q3. What is HNSW and why is it commonly used in vector search?

- A. Hierarchical Navigable Small World — a graph-based ANN index that provides fast and accurate nearest-neighbour search with good performance on high-dimensional data.**
- B. High-Noise Semantic Weighting — a technique for down-weighting noise in embedding vectors before indexing.
- C. Hybrid Neural Search Workflow — a pipeline combining dense and sparse retrieval in a single index.
- D. Horizontal Node Sharding and Weaving — a distributed architecture for scaling vector databases across multiple servers.

Correct Answer: A

Explanation

HNSW builds a hierarchical graph structure where each node connects to its nearest neighbours at multiple resolution levels. Search starts at the top (coarse) level and progressively refines — finding approximate nearest neighbours very quickly without exhaustive comparison. It is the dominant ANN algorithm in Pinecone, Weaviate, Chroma, and pgvector.

Q4. What does "cosine similarity" measure in the context of embedding search?

- A. The absolute distance between two vectors in Euclidean space.
- B. The number of shared tokens between two pieces of text after tokenisation.
- C. The angular similarity between two vectors — a value near 1 means the vectors point in the same direction, indicating semantic similarity.**
- D. The proportion of embedding dimensions where both vectors have the same sign.

Correct Answer: C

Explanation

Cosine similarity ignores vector magnitude and measures only direction — the angle between two vectors. In embedding space, direction encodes semantic meaning, so cosine similarity reliably captures semantic relatedness regardless of text length. Dot product is related and often used for efficiency when vectors are normalised.

Q5. What is the role of an embedding model in a RAG pipeline?

- A. Generating the final response from retrieved documents using the base language model.
- B. Converting text (queries and documents) into dense vectors so they can be compared for semantic similarity during retrieval.**
- C. Compressing retrieved documents into shorter summaries before they enter the context window.
- D. Classifying retrieved documents into topic categories to improve relevance ranking.

Correct Answer: B

Explanation

The embedding model is the semantic backbone of RAG. It converts both query text and document chunks into vectors in the same space. At retrieval time, query and chunk vectors are compared — chunks with high similarity to the query are retrieved. The quality of the embedding model directly determines retrieval quality.

Q6. What is the difference between "sparse" and "dense" retrieval?

- A. Sparse retrieval (BM25) uses keyword frequency and inverse document frequency; dense retrieval uses embedding similarity — combining both is often more effective than either alone.**
- B. Sparse retrieval works on small datasets; dense retrieval scales to large corpora.
- C. Sparse retrieval retrieves fewer documents; dense retrieval retrieves more documents per query.
- D. Sparse retrieval uses the full text; dense retrieval uses only the first sentence of each chunk.

Correct Answer: A

Explanation

Sparse retrieval (BM25, TF-IDF) matches based on exact term overlap and frequency — very effective for keyword queries. Dense retrieval (embedding similarity) captures meaning even without exact word match — effective for paraphrases and conceptual queries. Hybrid retrieval (combining both scores) typically outperforms either approach alone.

Q7. What is "metadata filtering" in a vector database and why is it important?

- A. Tagging embedding vectors with quality scores to filter out low-confidence retrievals.
- B. Removing metadata fields from documents before embedding to reduce vector noise.
- C. A post-processing step that filters retrieved documents by their cosine similarity score.
- D. Restricting vector search to documents matching specific metadata conditions (date, author, category) before or after ANN search — essential for access control and relevance filtering.**

Correct Answer: D

Explanation

Metadata filtering combines semantic search with structured constraints: "retrieve the most semantically similar chunks, but only from documents dated after 2024, tagged as financial reports, belonging to this user's organisation." Without metadata filtering, semantic search returns globally similar content, ignoring access permissions, recency, or domain scope.

Q8. What is "dimensionality reduction" in the context of embeddings and when is it used?

- A. Reducing the number of documents in an index by merging semantically similar chunks.
- B. Shrinking the embedding model's parameter count to reduce inference cost.
- C. Compressing high-dimensional embedding vectors into lower dimensions using techniques like PCA or UMAP — used to reduce storage cost, search latency, or to enable 2D/3D visualisation.**
- D. A technique for normalising embedding vectors to unit length before storage.

Correct Answer: C

Explanation

High-dimensional embeddings (1536D, 3072D) can be compressed to lower dimensions with modest accuracy loss. PCA reduces dimensions linearly preserving variance; UMAP preserves local structure for visualisation. Matryoshka embeddings (from OpenAI and others) are designed to be truncatable — you can use the first 256 dimensions of a 1536D embedding and still get reasonable retrieval quality.

Q9. What is "vector index freshness" and how is it managed?

- A. Keeping the vector index up to date as new documents are added or existing documents change — managed through re-indexing pipelines or real-time upsert operations.**
- B. The accuracy degradation of ANN indexes over time due to model drift.
- C. The frequency with which vector database providers update their ANN algorithms.
- D. A metric for how recently the embedding model was trained relative to the indexed documents.

Correct Answer: A

Explanation

A vector index is a snapshot. As source documents are added, updated, or deleted, the index must be kept in sync. This requires upsert pipelines (add new embeddings, update changed chunks, delete removed ones) and decisions about whether to re-index incrementally or rebuild periodically. Stale indexes return outdated or missing content.

Q10. What is "multi-tenancy" in a vector database and why does it matter for enterprise applications?

- A. The ability to run multiple embedding models simultaneously within the same index.
- B. A deployment pattern that runs the vector database across multiple cloud regions.
- C. A feature that allows multiple users to query the same index concurrently without performance degradation.
- D. The ability to store and search embeddings for multiple isolated organisations or users within a single database instance, with strict access separation.**

Correct Answer: D

Explanation

Enterprise RAG systems typically serve multiple customers or departments, each with private documents. Multi-tenancy ensures customer A's documents are invisible to customer B's queries. This is implemented through metadata filtering, namespace separation, or separate indexes per tenant — with different cost and isolation trade-offs.

Q11. What is "approximate" nearest-neighbour search and why is it used instead of exact search?

- A. ANN is used because exact search is mathematically impossible in high-dimensional spaces.
- B. ANN trades a small amount of accuracy for dramatically faster search — exact nearest-neighbour search over millions of vectors is too slow for real-time applications.**
- C. ANN returns approximate results to prevent the model from over-fitting to the retrieved content.
- D. ANN is a regulatory requirement for AI systems to prevent deterministic retrieval behaviour.

Correct Answer: B

Explanation

Exact nearest-neighbour search requires comparing the query vector against every indexed vector — $O(n)$ comparisons. Over millions of documents, this is too slow for real-time retrieval. ANN algorithms (HNSW, IVF) find vectors that are very likely nearest neighbours in sub-linear time, accepting a small accuracy trade-off for 100x or greater speed gains.

Q12. What is the significance of the embedding model's "embedding dimension" for a vector database?

- A. The embedding dimension determines the maximum document length that can be embedded.
- B. Higher embedding dimensions always improve retrieval accuracy regardless of the corpus size.
- C. The embedding dimension determines storage requirements and search latency — higher dimensions capture more semantic nuance but require more storage and compute for each comparison.**
- D. The embedding dimension must match the language model's hidden state size to work correctly.

Correct Answer: C

Explanation

Every embedded document costs (dimension $\times$ bytes per float) of storage. For 10M documents at 1536 dimensions in float32, that is ~60GB. Search latency also scales with dimension. Choosing embedding dimension involves balancing retrieval quality against infrastructure cost — Matryoshka embeddings allow post-hoc dimension reduction without re-embedding.

Q13. What is "namespace" or "collection" separation in a vector database?

- A. A feature that assigns human-readable names to individual embedding vectors for easier management.
- B. A partitioning technique that stores embeddings alphabetically by document title.
- C. A security mechanism that encrypts different vector partitions with different keys.
- D. Logically isolating groups of vectors within the same database so different applications or data sources can be searched independently.**

Correct Answer: D

Explanation

Namespaces (Pinecone) or collections (Chroma, Weaviate) allow a single vector database to serve multiple use cases: a customer support knowledge base in one namespace, a product catalogue in another, legal documents in a third. Queries target a specific namespace, preventing cross-contamination of retrieval results.

Q14. What is "re-ranking" after vector retrieval and why is it needed?

- A. A technique that re-orders retrieved documents chronologically before injecting them into context.
- B. A second-stage ranking step that uses a more accurate (but slower) model to re-score the top-k retrieved candidates, improving final result relevance.**
- C. A post-retrieval step that removes duplicate content from the retrieved candidate set.
- D. A method for re-embedding retrieved documents with a higher-quality model for final selection.

Correct Answer: B

Explanation

ANN retrieval is fast but imprecise — the top-20 results may include noise. Re-ranking applies a cross-encoder model (which processes query and document jointly, capturing interactions) to re-score and reorder the candidates. The top-5 after re-ranking are more precisely relevant than the top-5 from ANN alone, at the cost of additional latency.

Q15. What is "embedding drift" and when does it cause problems?

- A. When documents and queries are embedded with different model versions, their vectors are in incompatible spaces — retrieval quality degrades significantly.**
- B. The gradual degradation of embedding quality as the vector database becomes too large.
- C. The tendency for embedding models to produce less accurate vectors for rare vocabulary over time.
- D. A phenomenon where embeddings become less unique over time as new documents are indexed.

Correct Answer: A

Explanation

Embedding models have versions. If you embed your document corpus with model v1 and then upgrade to v2 for query embedding, the vectors are in different spaces — semantically similar items no longer cluster together. When upgrading embedding models, you must re-embed the entire corpus, not just new documents. Failing to do so causes silent retrieval failures.

SET

2

Chunking and Indexing

Set 2 of 5 · 15 Questions · RAG and Knowledge Systems

Q16. What is "chunking" in a RAG pipeline and why is it necessary?

- A. Compressing documents into dense summaries before embedding them.
- B. Dividing the vector index into shards for distributed storage across multiple servers.
- C. Splitting source documents into smaller segments that fit within the embedding model's token limit and provide focused, retrievable units of information.**
- D. Breaking the user's query into sub-questions to improve retrieval coverage.

Correct Answer: C

Explanation

Embedding models have token limits (typically 512–8192 tokens). Long documents cannot be embedded whole. More critically, a whole document embedded as one vector loses specificity — the embedding is a diffuse average of all its content. Chunking creates focused units: retrieving a specific relevant paragraph rather than a 100-page document.

Q17. What is "fixed-size chunking" and what is its main limitation?

- A. Splitting text into chunks of a fixed token count — simple to implement but can cut across sentence or paragraph boundaries, creating incoherent fragments.**
- B. Embedding each sentence as its own chunk — produces too many small chunks for efficient indexing.
- C. Dividing documents into fixed numbers of sections regardless of content length.
- D. A chunking method that preserves document structure by splitting only on heading markers.

Correct Answer: A

Explanation

Fixed-size chunking (e.g., 512 tokens) is the simplest approach: split every N tokens. The problem is arbitrary boundary cuts — a chunk might start mid-sentence or cut a key statement in half. Adding overlap (repeating the last 50 tokens in the next chunk) mitigates boundary issues but increases total chunk count and storage.

Q18. What is "semantic chunking" and what problem does it solve?

- A. Embedding each sentence independently and clustering similar sentences into a single chunk.
- B. Using the embedding model itself to determine the optimal chunk boundaries through similarity analysis.
- C. A chunking method that creates chunks based on the number of distinct concepts they contain.
- D. Splitting text at semantically meaningful boundaries — paragraphs, topic shifts, section headings — rather than at arbitrary token counts.**

Correct Answer: D

Explanation

Semantic chunking respects document structure: split at paragraph breaks, section headings, or topic change points detected by a small model. This produces chunks that contain coherent ideas rather than arbitrary text windows. The trade-off is higher implementation complexity and potentially uneven chunk sizes.

Q19. What is "chunk overlap" and what is its purpose?

- A. Storing multiple copies of the same chunk with different embeddings from different models.
- B. Repeating a portion of text at the boundary between adjacent chunks to prevent information loss at split points.**
- C. A technique for combining the embeddings of two adjacent chunks into a single vector.
- D. Overlapping metadata between chunks so retrieval systems can reconstruct document structure.

Correct Answer: B

Explanation

If a key concept straddles a chunk boundary, both halves of the statement are in adjacent chunks but neither half may be retrievable on its own. Overlap (repeating the last 100 tokens of chunk N as the first 100 tokens of chunk N+1) ensures boundary-straddling content is fully present in at least one chunk. It increases storage by the overlap percentage.

Q20. What is "hierarchical chunking" and when is it appropriate?

- A. Chunking a document tree structure based on its heading hierarchy.
- B. A method where each chunk is embedded at progressively lower dimensions for multi-resolution search.
- C. Creating chunks at multiple granularity levels — sentence, paragraph, section, document — and linking them, enabling retrieval at the right granularity for different query types.**
- D. Chunking documents in order of their importance, processing the most critical sections first.

Correct Answer: C

Explanation

Some queries need a single sentence; others need a full section. Hierarchical chunking indexes at multiple levels. Retrieval fetches the most relevant unit — perhaps a sentence — but the system can expand to the parent paragraph or section for full context before injection into the LLM prompt. This is the basis of "small-to-big retrieval" patterns.

Q21. What is a "document parser" in the context of RAG pipelines and why does it matter?

- A. A model component that classifies documents into categories before they are indexed.
- B. A tool that splits documents into chunks based on their natural language structure.
- C. A retrieval component that parses the user's query into structured search parameters.
- D. Software that extracts clean text from raw file formats (PDF, DOCX, HTML, spreadsheets) before chunking — poor parsing produces garbled chunks that degrade retrieval quality.**

Correct Answer: D

Explanation

Before chunking, documents must be converted from their native format to clean text. PDF parsing is notoriously unreliable — tables become garbled, columns merge, headers repeat. If the parsed text is poor, the embeddings are meaningless, and retrieval fails regardless of how good the rest of the pipeline is. High-quality parsing is a frequently underestimated RAG dependency.

Q22. What is "chunk size" and how should it be chosen?

- A. The token count of each chunk — chosen empirically based on the embedding model's limits, the granularity of information, and the typical query type, balancing specificity against context completeness.**
- B. The file size of the embedded chunk as stored in the vector database.
- C. The number of sentences per chunk, typically fixed at 3–5 for optimal retrieval.
- D. The token count must always match the context window size of the language model.

Correct Answer: A

Explanation

Chunk size is a hyper-parameter that affects retrieval quality. Small chunks (128 tokens) are specific but may lack context; large chunks (1024 tokens) are contextual but may dilute the embedding with irrelevant content. The right size depends on document type, query length, and embedding model — it should be determined through retrieval evaluation on real queries.

Q23. What is "parent-child retrieval" in a RAG system?

- A. A retrieval pattern where parent documents are retrieved first and child documents are retrieved from within them.
- B. Indexing small child chunks for precise retrieval, but injecting the larger parent chunk into context to provide surrounding information.**
- C. A hierarchical permission model where parent users can access all child users' retrieved documents.
- D. A chunking strategy where parent headings are stored as separate chunks above their child content.

Correct Answer: B

Explanation

Parent-child retrieval solves a trade-off: small chunks retrieve precisely (high recall for specific facts) but may lack context when injected into the LLM. The solution: index small chunks for retrieval, but when a chunk is selected, return its parent (the full paragraph or section) to the LLM. Specificity in retrieval, context in generation.

Q24. What is "document-level metadata" and how does it enhance retrieval?

- A. A summary of the document's content stored as a separate embedding for document-level retrieval.
- B. The document's file format and encoding information required by the parser.
- C. Metadata tags embedded within the chunk text to help the language model understand the source.
- D. Structured information about the document (author, date, source, category, access level) stored alongside chunk embeddings, enabling filtered and context-aware retrieval.**

Correct Answer: D

Explanation

Raw semantic search retrieves based on content alone. With metadata, you can constrain retrieval: "only documents from the legal department", "only published after Q3 2024", "only accessible to tier-1 users." This makes retrieval both more precise and more secure — essential for enterprise knowledge systems with access control requirements.

Q25. What is "index poisoning" and why is it a security concern for RAG systems?

- A. A database corruption attack that corrupts ANN index structures to cause retrieval failures.
- B. A technique used by attackers to extract documents from a vector database by observing query patterns.
- C. Injecting adversarial or misleading documents into the vector index to manipulate what the system retrieves and ultimately what the LLM says.**
- D. The gradual degradation of index quality caused by adding too many similar documents.

Correct Answer: C

Explanation

If an attacker can insert documents into your knowledge base — through a public data source, user-uploaded content, or web scraping — they can plant documents designed to be retrieved for specific queries. The LLM then incorporates this adversarial content into its response. Index poisoning is indirect prompt injection via the retrieval layer.

Q26. What is "sparse-dense hybrid indexing"?

- A. An indexing approach that stores small chunks densely and large chunks sparsely to optimise storage.
- B. Maintaining both a sparse keyword index (BM25) and a dense embedding index, then combining their scores for retrieval to get the benefits of both.**
- C. A technique that uses sparse matrix compression to reduce vector storage costs.
- D. An index that switches between sparse and dense retrieval based on query length.

Correct Answer: B

Explanation

Sparse indexes excel at exact term matching; dense indexes excel at semantic similarity. Hybrid indexes maintain both and combine scores (via reciprocal rank fusion or learned combination weights). This handles both keyword-specific queries ("ISO 27001 clause 6.2") and semantic queries ("what are our information security obligations") reliably.

Q27. What is "incremental indexing" and why does it matter operationally?

- A. Adding new documents to the index without rebuilding the entire index from scratch — critical for knowledge bases that receive frequent updates.**
- B. A technique for progressively improving index quality by re-embedding documents in order of retrieval frequency.
- C. Building the index in stages across multiple machines to reduce memory requirements.
- D. A method that incrementally compresses old embeddings to free space for new documents.

Correct Answer: A

Explanation

Many enterprise knowledge systems are dynamic — new documents added daily, old ones updated. Rebuilding the entire index from scratch for each update is too slow and expensive. Incremental indexing adds new documents with upsert operations (insert or update by document ID) without touching existing entries, keeping the index current without downtime.

Q28. What is "content-aware chunking" for structured documents like tables and code?

- A. A chunking technique that embeds the document's style and formatting alongside its text content.
- B. Using the document's own section markers to determine chunk boundaries.
- C. Treating tables, code blocks, and structured data as atomic units rather than splitting them mid-structure, preserving semantic integrity.**
- D. A method that assigns different embedding models to different content types within the same document.

Correct Answer: C

Explanation

Tables and code blocks cannot be split arbitrarily — a table split across two chunks loses its column-row structure; code split mid-function loses its meaning. Content-aware chunking detects these structures and treats them as atomic: a full table is one chunk, a complete function is one chunk — even if this violates the target token count.

Q29. What is a "knowledge graph" and how does it complement vector retrieval?

- A. A visual diagram of the vector space showing how document embeddings cluster.
- B. A database of factual triples used to verify the accuracy of LLM-retrieved content.
- C. A graph database version of a vector index that organises embeddings by topic hierarchy.
- D. A structured representation of entities and their relationships — useful for queries that require multi-hop reasoning across connected concepts that pure semantic search handles poorly.**

Correct Answer: D

Explanation

Vector retrieval is excellent at finding semantically similar content but poor at multi-hop reasoning: "What are the supply chain implications of our contract with Vendor X given our Q3 risk assessment?" requires traversing entity relationships. Knowledge graphs model entities and their connections explicitly, enabling reasoning that flat vector search cannot perform.

Q30. What is "late chunking" in modern RAG pipelines?

- A. Delaying the chunking step until query time, dynamically splitting documents based on query relevance.
- B. Embedding the full document first with a long-context embedding model, then splitting the contextualised token representations into chunk-level embeddings — preserving full-document context in each chunk's embedding.**
- C. A technique that embeds chunks only when they are first queried, rather than pre-computing all embeddings at index time.
- D. A post-retrieval step that re-splits retrieved chunks to fit the available context window.

Correct Answer: B

Explanation

Traditional chunking loses context: the embedding of a paragraph does not know about the rest of the document. Late chunking (pioneered by JinaAI) uses long-context embedding models to process the full document, then splits the resulting token-level representations into chunk embeddings. Each chunk embedding is contextualised by the entire document — improving retrieval for content that only makes sense in document context.

SET

3

Retrieval

Set 3 of 5 · 15 Questions · RAG and Knowledge Systems

Q31. What is "top-k retrieval" and what determines the right value of k?

- A. Returning the k most similar chunks to the query — k is chosen based on the context window budget, retrieval precision, and how much relevant content a typical query needs.**
- B. Retrieving all documents above a cosine similarity threshold, with k as the minimum floor.
- C. A retrieval strategy that always returns exactly 5 chunks regardless of query complexity.
- D. The k top-scored chunks from the sparse keyword index, used before dense re-ranking.

Correct Answer: A**Explanation**

Top-k is the most common retrieval pattern. The right k depends on: how much context window you can spare, how precise retrieval is (low precision means you need more candidates to ensure the right content is included), and the typical query complexity. k=3–5 is common for focused queries; k=10–20 for broad research queries with re-ranking.

Q32. What is "query expansion" in retrieval and how does it improve recall?

- A. Expanding the context window to accommodate more retrieved chunks.
- B. Breaking a complex query into sub-queries and retrieving results for each independently.
- C. Generating alternative phrasings or related terms for the query before retrieval, increasing the chance of matching relevant content that uses different vocabulary.**
- D. Adding metadata filters to a query after initial retrieval to narrow results.

Correct Answer: C**Explanation**

A user query uses specific vocabulary that may not match the vocabulary in relevant documents. Query expansion generates synonyms, related concepts, or re-phrasings: "heart attack" → also search "myocardial infarction", "cardiac event". This improves recall for vocabulary mismatches without reducing precision if done carefully.

Q33. What is "hypothetical document embedding" (HyDE) and what problem does it solve?

- A. Creating fictional document examples during index creation to improve embedding model training.
- B. Generating a hypothetical answer to the query, embedding that answer, and using it for retrieval — solving the vocabulary gap between short queries and longer document chunks.**
- C. Embedding a hypothetical user persona to personalise retrieval results.
- D. A technique for generating synthetic documents to fill gaps in a sparse knowledge base.

Correct Answer: B

Explanation

Short queries and long document chunks have very different linguistic characteristics — their embeddings may not be close even when semantically relevant. HyDE asks the LLM to generate a hypothetical answer to the query, then embeds that hypothetical answer for retrieval. The hypothetical answer is in the same register as real document chunks, improving embedding similarity.

Q34. What is "multi-query retrieval" and when is it useful?

- A. Sending the same query to multiple vector databases simultaneously and taking the union of results.
- B. Splitting a complex query into sequential sub-queries where each result informs the next.
- C. Retrieving from multiple indexes in parallel to compare coverage across different knowledge bases.
- D. Generating multiple reformulations of the user's query, retrieving results for each, and merging the results — useful for broad or ambiguous queries where a single phrasing misses relevant content.**

Correct Answer: D

Explanation

A single query phrasing may retrieve different content than a slightly different phrasing — due to vocabulary mismatch and the approximate nature of embedding similarity. Multi-query retrieval runs 3–5 reformulations and deduplicates the union of results. This improves coverage with modest latency overhead.

Q35. What is "retrieval precision" versus "retrieval recall" in a RAG context?

- A. Precision: proportion of retrieved chunks that are actually relevant; Recall: proportion of relevant chunks in the corpus that were successfully retrieved.**
- B. Precision: accuracy of the final LLM answer; Recall: proportion of the question that the answer addresses.
- C. Precision: speed of retrieval; Recall: completeness of the indexed corpus.
- D. Precision and recall are the same metric in retrieval contexts — both measure relevance.

Correct Answer: A

Explanation

These are the fundamental retrieval quality metrics. High precision means retrieved chunks are relevant (low noise in context); high recall means no important chunks were missed. Increasing k improves recall but may hurt precision. Re-ranking improves precision of the final selected chunks. The right trade-off depends on whether missing relevant content or including irrelevant content is more harmful.

Q36. What is "cross-encoder re-ranking" and how does it differ from bi-encoder retrieval?

- A. Cross-encoders retrieve from multiple embedding models simultaneously, taking the best result.
- B. Cross-encoders process query and document together in one pass, capturing interactions — more accurate than bi-encoders which encode them independently, but too slow for initial retrieval.**
- C. Cross-encoders use keyword overlap alongside semantic similarity for more accurate retrieval.
- D. Cross-encoders are a faster alternative to bi-encoders that sacrifice accuracy for speed.

Correct Answer: B

Explanation

Bi-encoders (used for initial retrieval) embed query and document separately — fast but limited by their inability to model query-document interactions. Cross-encoders take [query + document] as a single input and score the pair jointly — capturing nuanced relevance but requiring one forward pass per candidate. Used in the re-ranking stage on top-k candidates from bi-encoder retrieval.

Q37. What is "contextual retrieval" (as introduced by Anthropic) and what problem does it solve?

- A. A retrieval technique that uses the conversation history as additional context for query reformulation.
- B. A method where retrieved chunks are summarised before being injected into the LLM context.
- C. A retrieval approach that retrieves both the relevant chunk and its adjacent chunks automatically.
- D. Prepending a chunk-specific summary of its role within the parent document to each chunk before embedding — solving the problem of chunks that lose meaning without document context.**

Correct Answer: D

Explanation

Anthropic's contextual retrieval addresses a core problem: a chunk saying "The agreement was signed on 15 March 2024" is meaningless without knowing which agreement. Contextual retrieval uses the LLM to generate a brief description of what the chunk covers in the context of the full document, prepends this to the chunk before embedding, and improves retrieval accuracy significantly.

Q38. What is "retrieval augmented generation" (RAG) at its most fundamental level?

- A. Training a language model on retrieved web content to keep its knowledge current.
- B. Using a language model to generate improved search queries for a traditional search engine.
- C. Retrieving relevant external content at query time and injecting it into the LLM context, grounding the response in provided information rather than the model's parametric knowledge alone.**
- D. A technique for compressing retrieval results into the model's context window more efficiently.

Correct Answer: C

Explanation

RAG is a two-step architecture: retrieve then generate. Step 1: convert the query to an embedding, search the vector index, and retrieve relevant chunks. Step 2: inject those chunks into the LLM prompt alongside the query. The LLM generates a response grounded in the retrieved content rather than from parametric memory alone. This is the foundation of almost all enterprise AI knowledge applications.

Q39. What is "self-querying retrieval" and how does it improve structured knowledge bases?

- A. A retrieval pattern where the model generates its own retrieval queries without user input.
- B. Using an LLM to convert natural language queries into structured filters for the vector database — combining semantic search with metadata filtering based on the user's intent.**
- C. A method where the retrieval system queries itself to identify gaps in the knowledge base.
- D. A technique for retrieving documents that match the style of the user's query rather than its content.

Correct Answer: B

Explanation

Self-querying retrieval uses an LLM to parse the query into a semantic component and a structured filter component: "documents about quarterly performance from Q1 2025" → semantic query = "quarterly performance" + filter = "date between 2025-01-01 and 2025-03-31." The LLM generates both, enabling natural language queries against structured metadata.

Q40. What is "reciprocal rank fusion" (RRF) and when is it used?

- A. A method for combining ranked result lists from multiple retrieval systems by aggregating reciprocal rank positions — used in hybrid retrieval to merge sparse and dense results.**
- B. A technique for re-ranking retrieved documents based on their relevance to the full conversation history rather than just the current query.
- C. A scoring method that multiplies the dense retrieval score by the sparse retrieval score.
- D. A fusion technique that averages the embedding vectors of top retrieved chunks before LLM injection.

Correct Answer: A

Explanation

When combining results from BM25 and dense retrieval, you have two ranked lists with different score scales. RRF converts each list to ranks and combines them: $\text{score} = \sum 1/(k + \text{rank}_i)$ where k is a smoothing constant. This is robust to score scale differences and consistently outperforms naive score averaging in hybrid retrieval benchmarks.

Q41. What is "retrieval latency" and why does it matter for user-facing applications?

- A. The delay between a document being added to the corpus and it becoming retrievable.
- B. The time taken for the LLM to process retrieved chunks and generate a response.
- C. The time taken from query submission to when retrieved chunks are available for the LLM — in user-facing applications, retrieval latency adds directly to perceived response time.**
- D. A metric for how quickly the vector index can be rebuilt after a major update.

Correct Answer: C

Explanation

For real-time applications, every millisecond matters. Retrieval latency depends on: index size, ANN algorithm, k , whether re-ranking is applied, and database hardware. Caching frequent queries, using in-memory indexes, and limiting re-ranking candidates are common optimisations. A slow retrieval step can make an otherwise fast LLM feel sluggish.

Q42. What is "query routing" in a multi-knowledge-base RAG system?

- A. Load-balancing retrieval queries across multiple vector database replicas.
- B. A technique for splitting a complex query into multiple sub-queries routed to different LLMs.
- C. Forwarding unanswerable queries to a human agent after retrieval fails.
- D. Classifying incoming queries and directing them to the most appropriate knowledge base or retrieval strategy based on query type or domain.**

Correct Answer: D

Explanation

Large organisations may have multiple knowledge bases: HR policies, technical documentation, product specifications, legal contracts. Query routing classifies the intent and directs to the right source: "questions about leave policy" → HR index; "questions about API specs" → technical docs. This improves both precision and retrieval efficiency compared to searching all indexes for every query.

Q43. What is "passage retrieval" versus "document retrieval"?

- A. Passage retrieval uses embedding similarity; document retrieval uses keyword matching.
- B. Passage retrieval returns the specific relevant section within a document; document retrieval returns the whole document — passage retrieval is more precise for focused queries.**
- C. Passage retrieval is used for short documents; document retrieval is used for long documents.
- D. Passage and document retrieval are equivalent — the distinction is in how results are displayed, not how they are retrieved.

Correct Answer: B**Explanation**

Document retrieval returns whole documents — useful when the entire document is relevant or when document-level context is needed.

Passage retrieval returns specific sections — more efficient for focused queries and better preserves context window budget. Most modern RAG systems use passage (chunk) retrieval, with optional parent document expansion for additional context.

Q44. What is "negative sampling" in embedding model training and why does it matter for retrieval quality?

- A. Removing low-quality documents from the index to prevent them from being retrieved.
- B. A technique for generating synthetic queries for documents that have no natural associated queries.
- C. Training the embedding model on hard negative examples — documents that are superficially similar to relevant ones but should be ranked lower — to produce embeddings that distinguish near-duplicates.**
- D. Sampling a subset of the corpus to train a small retrieval model for low-resource settings.

Correct Answer: C**Explanation**

Hard negatives are the key to high-quality retrieval embeddings. If the model only trains on obvious positives and random negatives, it learns coarse distinctions. Hard negatives — documents that seem relevant but are not — force the model to learn fine-grained distinctions. Models trained with hard negatives (like E5, BGE, and mxbai-embed) significantly outperform those trained without them.

Q45. What is "retrieval evaluation" and what metrics are used?

- A. Systematic measurement of retrieval quality using metrics like recall@k, precision@k, NDCG, and MRR against a labelled dataset of queries and relevant documents.**
- B. An evaluation of the LLM's ability to use retrieved context accurately in its final answer.
- C. A benchmark test that measures how quickly the vector database returns results under load.
- D. An audit of the knowledge base for completeness and accuracy before deploying a RAG system.

Correct Answer: A

Explanation

Retrieval evaluation requires a golden dataset: queries with known relevant documents. Key metrics: recall@k (are relevant docs in the top-k?), precision@k (how many of the top-k are relevant?), NDCG (rank-weighted precision), MRR (rank of the first relevant result). Without retrieval evaluation, you cannot tell if changes to chunking, embedding models, or retrieval strategy improve or hurt quality.

SET

4

Grounding and Hallucination Prevention

Set 4 of 5 · 15 Questions · RAG and Knowledge Systems

Q46. What is "grounding" in a RAG system and why is it the primary mechanism for reducing hallucination?

- A. Connecting the model to real-time data sources so its knowledge is always current.
- B. A technique for fact-checking model outputs against a database of verified facts after generation.
- C. Anchoring the model's outputs in user-specific context by injecting user profile data.
- D. Instructing the model to base its response on provided documents only, making fabricated claims verifiable and constraining the model away from its parametric knowledge.**

Correct Answer: D**Explanation**

Grounding works by replacing the question "what do I believe?" with "what does the provided context say?" When the model is instructed to answer only from retrieved documents, every claim traces back to a specific source. This does not eliminate hallucination (models can still misread sources) but dramatically reduces fabrication of facts not in the provided content.

Q47. What is "faithfulness" as a RAG evaluation metric?

- A. Whether the retrieved documents are factually correct and up to date.
- B. Whether the model's response accurately reflects the content of the retrieved documents, without adding claims not supported by the retrieved context.**
- C. Whether the model responds consistently across multiple runs of the same query.
- D. Whether the model's persona remains consistent with the system prompt throughout the conversation.

Correct Answer: B**Explanation**

Faithfulness measures the alignment between retrieved context and generated response. A faithful response makes only claims that are supported by the retrieved chunks. An unfaithful response adds information from the model's parametric knowledge — which may or may not be accurate. Faithfulness can be measured with LLM-as-judge evaluation against the retrieved source.

Q48. What is "answer relevance" in RAG evaluation and how does it differ from faithfulness?

- A. Answer relevance measures output length; faithfulness measures citation accuracy.
- B. Answer relevance is an automated metric; faithfulness requires human evaluation.
- C. Answer relevance measures whether the response addresses the user's question; faithfulness measures whether it is grounded in the retrieved context — a response can be faithful but irrelevant, or relevant but unfaithful.**
- D. Answer relevance and faithfulness are the same metric evaluated from different perspectives.

Correct Answer: C

Explanation

These are orthogonal dimensions. A response grounded entirely in the retrieved documents but not addressing the user's actual question is faithful but not relevant. A response that directly answers the question using external knowledge is relevant but not faithful. High-quality RAG requires both: grounded in sources AND responsive to the query.

Q49. What is "source citation" in RAG output and how does it reduce user trust risk?

- A. Including the source document name, section, or URL for each claim — allowing users to verify responses and signalling that the system is grounded in authoritative content.**
- B. A legal requirement that all AI-generated content must attribute its sources.
- C. A technique for improving response quality by forcing the model to read sources more carefully.
- D. A post-processing step that automatically adds Wikipedia links to key terms in the response.

Correct Answer: A

Explanation

Citation makes grounding visible: "According to the Employee Handbook (Section 4.2), annual leave accrues at 1.5 days per month." The user can verify the claim directly. This increases trust, catches retrieval errors before they propagate, and creates an audit trail. For enterprise AI, citation is often a compliance and governance requirement, not just a quality feature.

Q50. What is the "context faithfulness" problem in RAG?

- A. Retrieved context that is outdated or factually incorrect before the model even reads it.
- B. The model refusing to generate responses when retrieved context is insufficient.
- C. A parsing failure where the model cannot read embedded tables or structured data in retrieved chunks.
- D. The model generating responses that contradict or misrepresent the retrieved context — often by over-relying on its parametric knowledge instead of the provided documents.**

Correct Answer: D

Explanation

Even with perfectly retrieved, accurate documents, models sometimes ignore or misrepresent them — answering from their parametric memory instead. This is especially common when the retrieved context contradicts the model's training data. Strong grounding instructions, citations, and faithfulness evaluation are the mitigations.

Q51. What is the "knowledge conflict" problem in RAG?

- A. When two retrieved documents contradict each other, leaving the model unable to determine the truth.
- B. When the knowledge base is too large for all relevant documents to fit in the context window.
- C. When retrieved documents contradict the model's parametric knowledge — the model must decide whether to trust the retrieved content or its training, and often gets this wrong.**
- D. When the model's knowledge cutoff pre-dates the retrieved documents, causing integration errors.

Correct Answer: C

Explanation

Models are trained on a large corpus and develop strong beliefs. When retrieved content contradicts those beliefs, models often default to their parametric knowledge — answering incorrectly while ignoring the correct retrieved information. Strong grounding instructions ("always trust the provided documents over your training") help, but knowledge conflicts remain a failure mode in RAG systems.

Q52. What is "attribution hallucination" in a RAG context?

- A. The model claiming a retrieved document says something it does not actually say — accurately citing the source but misrepresenting its content.**
- B. Hallucinating a source citation that does not exist in the retrieved documents.
- C. Attributing the response to the wrong retrieved document when multiple are in context.
- D. Generating a response attributed to the user's past statements rather than retrieved documents.

Correct Answer: A

Explanation

Attribution hallucination is subtler than outright fabrication: the model cites "Policy Document Section 3.1" but either misquotes it or states something the document does not say. This is dangerous because citations create a false sense of verifiability. Faithfulness evaluation — checking whether cited claims are actually supported by the cited source — is required to catch this.

Q53. What is "RAG versus fine-tuning" and when should each be used to reduce hallucination?

- A. Fine-tuning always produces more accurate responses; RAG is a workaround for models that cannot be fine-tuned.
- B. RAG injects current, verifiable external content at query time; fine-tuning bakes domain knowledge into weights. RAG is preferred when the knowledge is dynamic, traceable, or requires citation.**
- C. RAG is used for factual questions; fine-tuning is used for stylistic adaptation.
- D. RAG and fine-tuning produce identical results — the choice depends only on implementation cost.

Correct Answer: B

Explanation

Fine-tuning for factual knowledge is unreliable — models can still hallucinate domain facts even after fine-tuning, and the knowledge cannot be updated without re-training. RAG provides verifiable, updateable, citable knowledge at query time. Fine-tuning is better for style, format, and task behaviour — not for injecting facts that need to be trusted.

Q54. What is "abstention" and when should a RAG system abstain?

- A. A safety mechanism that prevents the model from generating responses on sensitive topics.
- B. A retrieval strategy that returns no results when confidence is below a threshold.
- C. Declining to answer when the retrieved context does not contain sufficient information, rather than hallucinating a response.**
- D. A technique where the model refers users to human agents instead of answering.

Correct Answer: C

Explanation

A well-designed RAG system should abstain gracefully: "I don't have sufficient information in the available documents to answer this question." This is far better than hallucinating an answer. Abstention requires explicit instruction and a faithfulness evaluation step — without it, models default to generating plausible-sounding content rather than admitting uncertainty.

Q55. What is "chunk context contamination" and how does it cause incorrect responses?

- A. A corruption where encoding errors in retrieved chunks introduce garbled text into the context.
- B. The injection of adversarial content into retrieved chunks by a malicious document author.
- C. A problem where chunks from different documents are combined in a way that creates copyright issues.
- D. When irrelevant or misleading retrieved chunks are included in the context alongside relevant ones, and the model synthesises information across all of them, producing a confused response.**

Correct Answer: D

Explanation

If retrieval returns 5 chunks but 3 are irrelevant, the model may blend information from all 5. A question about the termination clause in Contract A might be answered with information from Contract B that was co-retrieved. This is why retrieval precision matters: reducing irrelevant chunks reduces contamination risk, not just noise.

Q56. What is "confidence-based routing" in a RAG system?

- A. Routing queries to different response strategies based on retrieval confidence — high-confidence retrievals go to the LLM; low-confidence ones trigger clarification, abstention, or human escalation.**
- B. A technique for routing high-traffic queries to faster model endpoints during peak load.
- C. A method for automatically selecting the most confident retrieved chunk as the sole context.
- D. A routing system that sends low-confidence responses to a human reviewer before delivery.

Correct Answer: A

Explanation

Not all retrievals are equally reliable. A retrieval score above a threshold indicates good matches exist in the knowledge base; below the threshold, the knowledge base may not cover the query. Confidence-based routing handles this: certain responses are answered directly, uncertain ones prompt clarification or escalation, preventing unreliable responses from reaching users.

Q57. What is "closed-book versus open-book" generation in LLM systems?

- A. Closed-book models cannot access the internet; open-book models can browse the web in real time.
- B. Closed-book: the model answers from parametric memory only; Open-book (RAG): the model answers from provided documents — open-book is generally more accurate for specific factual questions.**
- C. Closed-book generation requires fine-tuning; open-book generation uses the base model.
- D. Closed-book and open-book refer to whether the model's system prompt is visible to users.

Correct Answer: B

Explanation

The closed/open-book distinction captures whether the model has access to external context. Closed-book models answer from weights alone — fast, private, but limited to training knowledge. Open-book (RAG) models answer from provided documents — more accurate for specific, current, or proprietary knowledge, but dependent on retrieval quality.

Q58. What is the "faithfulness versus completeness" trade-off in RAG responses?

- A. Faithful responses are shorter; complete responses are longer — the trade-off is response length.
- B. Faithfulness is a user-facing quality; completeness is an internal system metric.
- C. The trade-off only applies when retrieved content is from multiple conflicting sources.
- D. Maximally faithful responses stick close to retrieved content and may miss information not in the context; complete responses may hallucinate to fill gaps — the balance depends on application risk tolerance.**

Correct Answer: D

Explanation

Faithful responses say only what the sources say — they may be incomplete if the sources do not cover the full answer. Complete responses try to answer the whole question — they may add parametric knowledge, risking hallucination. For high-stakes applications (legal, medical, compliance), faithfulness dominates. For research and exploration, completeness may be more valuable.

Q59. What is "hallucination detection" as a post-generation step?

- A. Running a secondary check — using an LLM or a fact-checking model — to identify claims in the response that are not supported by the retrieved context.**
- B. A pre-generation filter that blocks prompts likely to cause hallucination.
- C. A metric computed from the model's internal token probability distributions.
- D. A technique for identifying hallucinated citations by querying the source database.

Correct Answer: A

Explanation

Post-generation hallucination detection runs a verification pass: "For each claim in this response, is it supported by one of the retrieved documents? Flag any claim that is not." This can be automated with LLM-as-judge prompting against the retrieved context. It adds latency and cost but is essential for high-stakes applications where undetected hallucination is unacceptable.

Q60. What is a "grounding score" and how is it used in RAG evaluation?

- A. A score assigned to each retrieved chunk indicating how relevant it is to the query.
- B. A benchmark score that compares RAG system performance against non-RAG baselines.
- C. A metric that measures what proportion of the response's claims are traceable to the retrieved context — used to monitor and improve RAG faithfulness over time.**
- D. A confidence score generated by the LLM indicating how certain it is about its response.

Correct Answer: C

Explanation

Grounding score = (claims supported by retrieved context) / (total claims in response). Tracking this metric across queries identifies patterns: certain query types cause more hallucination, certain document types are misread, or certain system prompt formulations improve grounding. It is an actionable metric for systematic RAG quality improvement.

SET

5

Enterprise Knowledge Systems

Set 5 of 5 · 15 Questions · RAG and Knowledge Systems

Q61. What is an "enterprise knowledge system" built on RAG?

- A. An AI application that connects organisational knowledge sources — policies, documents, databases, wikis — to a language model via retrieval, enabling employees to query internal knowledge in natural language.
- B. A database system that uses AI to automatically categorise and tag enterprise documents.
- C. A model fine-tuned on an organisation's proprietary data to answer internal questions.
- D. A search engine that uses keyword indexing to retrieve enterprise documents faster than traditional search.

Correct Answer: A**Explanation**

Enterprise knowledge systems are the most common enterprise AI application: employees ask questions in natural language ("What is our reimbursement policy for client entertainment?") and the system retrieves the relevant policy document and generates a grounded, cited response. The quality depends on parsing, chunking, retrieval, and grounding — not just the LLM.

Q62. What is "access control" in an enterprise RAG system and how is it typically implemented?

- A. Encrypting all documents in the vector index to prevent unauthorised access to raw embeddings.
- B. A login system that authenticates users before allowing them to query the knowledge base.
- C. A post-retrieval filter that removes sensitive content from responses before delivery.
- D. Ensuring users can only retrieve content they are authorised to see — typically through metadata filtering by user role, department, or document classification.

Correct Answer: D**Explanation**

In an enterprise context, not all employees should see all documents — financial forecasts, HR records, legal privileged content. Access control in RAG is typically implemented via metadata: each document is tagged with access permissions, and retrieval queries are filtered to include only documents the requesting user is authorised to view. This must be enforced at the database layer, not just in the prompt.

Q63. What is "knowledge freshness" and how is it managed in a production RAG system?

- A. Using a recent embedding model to ensure indexed documents reflect the latest language patterns.
- B. Keeping the indexed knowledge current by continuously ingesting new documents, updating changed ones, and removing deprecated ones through automated pipelines.**
- C. Restricting retrieval to documents published within the last 90 days to prevent stale answers.
- D. Retraining the language model periodically to incorporate knowledge from the organisation's documents.

Correct Answer: B

Explanation

A knowledge base that is not updated becomes a liability: employees get answers based on outdated policies, superseded product specs, or revoked legal guidance. Knowledge freshness requires document ingestion pipelines that detect new or modified source documents, re-chunk and re-embed them, upsert into the vector index, and delete removed documents.

Q64. What is a "knowledge gap" in an enterprise RAG system and how should it be handled?

- A. A discrepancy between what the LLM knows parametrically and what is in the knowledge base.
- B. A retrieval failure caused by poor chunking that prevents the system from finding existing content.
- C. When the knowledge base does not contain information to answer a query — handled by abstaining gracefully, routing to a human expert, or flagging the gap for knowledge base improvement.**
- D. The difference in quality between documents produced by different departments in the organisation.

Correct Answer: C

Explanation

No knowledge base is complete. A well-designed system detects when it cannot find relevant content (low retrieval scores, no above-threshold results) and responds appropriately: "I don't have documentation on this topic. Please contact [expert team]." This is more professional than hallucinating an answer. Tracking knowledge gaps also provides a prioritised backlog for knowledge base improvement.

Q65. What is "document lifecycle management" in the context of RAG?

- A. Tracking documents through creation, versioning, expiry, and deletion — ensuring the knowledge base reflects current authoritative versions and removes outdated content.**
- B. A content governance process for ensuring documents are written in a RAG-compatible format.
- C. A backup and recovery system for vector database contents.
- D. The process of archiving retrieved responses for audit and compliance purposes.

Correct Answer: A

Explanation

Documents have lifecycles. Policies are superseded; product specs are updated; contracts expire. Without lifecycle management, a RAG system might retrieve an outdated policy that has been replaced. This requires: version control (new versions replace old), expiry metadata (auto-remove content after a date), and governance processes to ensure the knowledge base reflects the current authoritative state.

Q66. What is "semantic deduplication" in knowledge base curation?

- A. A technique for removing identical files from a document repository before indexing.
- B. Using an LLM to merge similar documents into a single authoritative version.
- C. Identifying and removing near-duplicate documents or chunks using embedding similarity — preventing the same information from being retrieved multiple times and inflating its apparent importance.**
- D. A post-retrieval step that removes duplicate chunks before injecting them into the LLM context.

Correct Answer: C

Explanation

Document repositories accumulate duplicates: slightly different versions of the same policy, copied content in multiple documents, email threads that quote each other. Semantic deduplication identifies near-duplicate chunks using cosine similarity and removes or merges them. Duplicate content biases retrieval — the same information retrieved multiple times appears more relevant than it is.

Q67. What is "knowledge base segmentation" and when is it appropriate?

- A. Splitting large documents into smaller ones to improve chunking quality.
- B. A technique for distributing the vector database across multiple servers for performance.
- C. Segmenting users into groups that receive different knowledge base access for A/B testing.
- D. Organising the knowledge base into separate indexes or namespaces by domain, department, or topic — improving retrieval precision and enabling domain-specific access control.**

Correct Answer: D

Explanation

A single global knowledge base for an entire enterprise mixes HR policies, technical docs, legal agreements, and marketing collateral. Retrieving "salary" in this context might return both compensation policy and a sales script. Segmentation into domain-specific indexes improves precision, enables department-specific access control, and allows different chunking/embedding strategies per domain.

Q68. What is "RAG observability" and what does it involve?

- A. A technique for making the RAG system's internal workings visible to end users.
- B. Monitoring and logging the full RAG pipeline — queries, retrieved chunks, generation inputs and outputs — to diagnose failures, measure quality, and identify improvement opportunities.**
- C. Real-time monitoring of vector database performance metrics like latency and throughput.
- D. An audit trail of all user queries for compliance and legal discovery purposes.

Correct Answer: B

Explanation

RAG systems fail in many ways: retrieval misses relevant content, retrieved content is misread, grounding fails, answers hallucinate. Without observability — logging queries, retrieved chunks, relevance scores, and generated responses — diagnosing these failures is impossible. Full pipeline logging enables root cause analysis, quality metric tracking, and targeted improvement.

Q69. What is "hybrid knowledge architecture" combining structured and unstructured data?

- A. A database architecture that uses different storage formats for different document types.
- B. A retrieval system that uses different embedding models for different content types.
- C. An AI system that retrieves from both vector databases (for semantic search of unstructured text) and relational databases (for precise structured data queries) and synthesises both in the response.**
- D. A knowledge base that mixes publicly available and proprietary documents.

Correct Answer: C

Explanation

Real enterprise queries often require both: "What does our policy say about contractor expenses AND what was our actual contractor spend last quarter?" The first is unstructured (vector search); the second is structured (SQL). Hybrid architectures retrieve from both, synthesising a unified response. This is significantly more complex than pure RAG but reflects real enterprise information needs.

Q70. What is "personalised retrieval" in an enterprise context?

- A. Adapting retrieved content based on the user's role, preferences, history, or current project context — surfacing the most relevant content for that specific user rather than generic results.**
- B. A technique for retrieving information that matches the user's writing style.
- C. A retrieval strategy that prioritises documents the user has previously accessed.
- D. Adjusting retrieval scores based on the user's department to apply access control.

Correct Answer: A

Explanation

Generic retrieval returns the same results for the same query regardless of who is asking. A sales manager asking about "product performance" needs different content than a product engineer asking the same question. Personalised retrieval uses the user's role, recent activity, and current projects to adjust retrieval — returning contextually relevant results rather than globally relevant ones.

Q71. What is "knowledge base quality scoring" and why is it important before RAG deployment?

- A. A technique for benchmarking the knowledge base's retrieval performance against industry standards.
- B. Assessing the coverage, accuracy, currency, and format quality of documents in the knowledge base before deployment — poor quality input produces poor quality RAG output.**
- C. Automatically scoring document importance to prioritise which content to index first.
- D. A compliance assessment of knowledge base content for regulatory requirements.

Correct Answer: B

Explanation

Garbage in, garbage out. A RAG system is only as good as its knowledge base. Pre-deployment quality scoring assesses: coverage (are the key topics represented?), accuracy (is the content correct?), currency (is it up to date?), and format quality (is it cleanly parsed and well-structured?). Deploying RAG over a poor knowledge base delivers poor results regardless of model quality.

Q72. What is "chunking for tables" and why does it require special handling?

- A. Tables should always be embedded as plain text with whitespace preserved to maintain alignment.
- B. Tables in documents should be removed before indexing because they confuse embedding models.
- C. Table cells should each be indexed as separate chunks to maximise retrieval granularity.
- D. Tables must be preserved as complete atomic units or converted to structured formats, because splitting a table mid-row destroys the row-column relationships needed to interpret the data.**

Correct Answer: D

Explanation

A table split across chunks loses the header-row relationship: the cell "42%" on its own is meaningless without the column header "Q3 Revenue Growth" and the row label "APAC Region." Tables should be extracted whole, either embedded as serialised text (Markdown tables, CSV-like text) or stored as structured data with metadata describing each cell's row and column context.

Q73. What is a "retrieval-augmented fine-tuning" (RAFT) approach?

- A. Fine-tuning an embedding model on domain-specific query-document pairs to improve retrieval quality.
- B. A technique for fine-tuning the entire RAG pipeline jointly — both retriever and generator — on task-specific data.
- C. Fine-tuning the language model to be better at using retrieved documents — specifically training it to identify relevant context, ignore distractors, and extract answers faithfully.**
- D. Using RAG to generate training data for fine-tuning a smaller, more efficient model.

Correct Answer: C

Explanation

RAFT (proposed by researchers at Berkeley) fine-tunes the language model specifically for RAG tasks: it trains the model to read a document set containing relevant and irrelevant "distractor" documents, identify which parts are relevant, and produce a faithful answer with citations. The result is a model that is better at the RAG task than a general-purpose model.

Q74. What is the "two-stage RAG" architecture and what problem does it solve?

- A. An architecture with two separate knowledge bases — one for retrieval and one for grounding validation.
- B. A retrieve-then-filter architecture: first retrieve a large candidate set (high recall), then apply a more expensive filter or re-ranker to select the final context (high precision) — solving the precision-recall trade-off.**
- C. A two-model architecture where one model retrieves and another model generates the response.
- D. A pipeline that runs two independent RAG calls and merges their responses.

Correct Answer: B

Explanation

Single-stage retrieval must balance recall (retrieve enough relevant content) and precision (avoid noise). Two-stage RAG separates these: stage 1 retrieves broadly (top-50 ANN results, high recall), stage 2 applies a cross-encoder re-ranker to select the best 5 (high precision). The additional latency from re-ranking is offset by significantly better final context quality.

Q75. What is "RAG evaluation at the pipeline level" and what does it measure?

- A. A performance test that measures the throughput and latency of the production RAG pipeline.
- B. An audit of the RAG system's compliance with data governance policies.
- C. Evaluating the quality of individual components (chunker, embedder, retriever) separately before integration.
- D. Measuring the end-to-end performance of the complete RAG system — retrieval quality, grounding faithfulness, and answer accuracy — rather than evaluating each component in isolation.**

Correct Answer: D

Explanation

Component-level evaluation (retrieval recall, embedding quality, LLM faithfulness) can all look good while the end-to-end system performs poorly due to interactions between components. Pipeline-level evaluation measures what users actually experience: given a set of representative questions, does the system return correct, grounded, complete answers? This is the ultimate quality signal.

END OF QUESTION BANK

*Your turning
point starts
here.*